Game Shop Chatbot: Interactive NLP-Based Chatbot

Game Shop Chatbot

Interactive NLP-based system report for COMP3074 Human-AI Interaction

ALI MERT KARAOGLU
University of Nottingham, Computer Science, psyak24@nottingham.ac.uk

1 Introduction

With ongoing advancements in artificial intelligence and computer science, NLP technologies have evolved significantly and are now widely used in everyday digital tasks [1]. These technologies offer users flexible and accessible interaction through natural-language dialogue [2]. From voice assistants to customer service chatbots, conversational agents are improving automation and are reshaping how people access information and complete online transactions [3]. Building on this context, this report describes the development of a rule-driven, NLP-based chatbot designed to help users browse and purchase video games.

The aim of this project is to develop a text-based conversational assistant for a video game shop scenario, where users can discover, compare and purchase games through natural language rather than by navigating a traditional graphical interface. The chatbot acts as a virtual shop assistant that supports small talk, answers domain-specific questions, and guides users through a multi-step game selection and purchasing flow based on their preferences and chosen platform. To support this, the system relies on a rule-driven intent classification pipeline combined with TF-IDF vectorisation and cosine similarity, rather than using external frameworks such as LLMs or transformer-based models. This helps to keep the interaction logic simple and manageable for the specific task of helping users with video game purchases.

At a high level, the system follows a modular pipeline with five main stages: preprocessing, text representation, similarity-based intent classification, dialogue management, and template-based response generation. The remainder of this report first describes this architecture in more detail, then discusses the conversational design choices and evaluation results, and finally reflects on the implications and limitations of the chatbot within a video game shop context.

2 Chatbot Architecture

The system adopts a modular architecture in which user inputs flow through a pipeline of vectorisation, intent classification, intent handling, and natural language generation. The intent handler acts as the central control unit of the chatbot, coordinating all specialised components including the intent, name, question-answering (QA), and game modules, as shown in Figure 1.

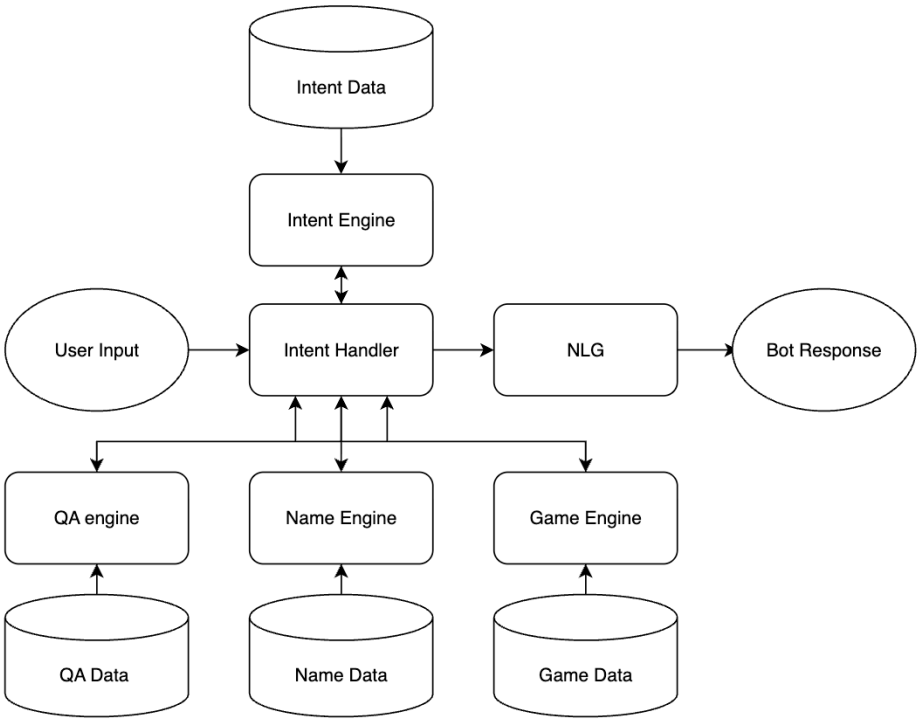


Figure 1: High-level System Architecture of the Chatbot

2.1 Functionality

2.1.1 Identity Management

The chatbot supports identity management by storing and recalling the user's name. Users can provide, update, or remove their stored name. The chatbot personalises all responses using the stored name and allows users to control whether their name persists between sessions.

2.1.2 Small Talk and Emotional Awareness

The chatbot handles small-talk utterances, including greetings and casual questions, and supports these interactions by identifying positive or negative sentiment in user input to generate emotionally appropriate responses.

2.1.3 Question Answering

The chatbot can answer questions about domain-specific topics. When the user asks a question, the system searches its knowledge base for the most relevant answer. If multiple answers are available, it randomly selects one.

2.1.4 Help and Discoverability

When users request help, the system displays a comprehensive list of capabilities, including examples of how to browse, filter, and purchase games step-by-step, along with guidance on identity management and small talk.

2.1.5 Game Browsing and Search

The games handler enables users to search and filter games across multiple attributes. Users can:

- Search by title to find specific games
- Filter by platform
- Filter by genre
- Filter by price
- Filter by critic score
- Filter by release date
- Browse games by developer or publisher
- View extremes: cheapest/most expensive, lowest/highest rated, oldest/newest games
- Explore all available platforms and genres in the database

These operations can be combined over multiple turns, allowing users to refine their search interactively. The chatbot also supports simple game recommendations by suggesting titles that match the genre and platform of the user's most recent purchase.

2.1.6 History Management

The system maintains both search and purchase history. Users may review their previous queries as a numbered list, rerun any item, or clear the list entirely. Purchase history can similarly be retrieved or deleted when requested.

2.1.7 Purchase Flow

When a user decides to purchase a game, the system initiates a structured transaction dialogue (Figure 2). It first retrieves games matching the user's input. If multiple titles match, the user selects one from a numbered list. If the chosen game is available on multiple platforms, the chatbot presents the platform options and corresponding prices, and asks the user to select one. The system then summarises the order (title, platform, and price) and requests explicit confirmation. Users can exit the purchase flow at any point using a dedicated "back" intent.

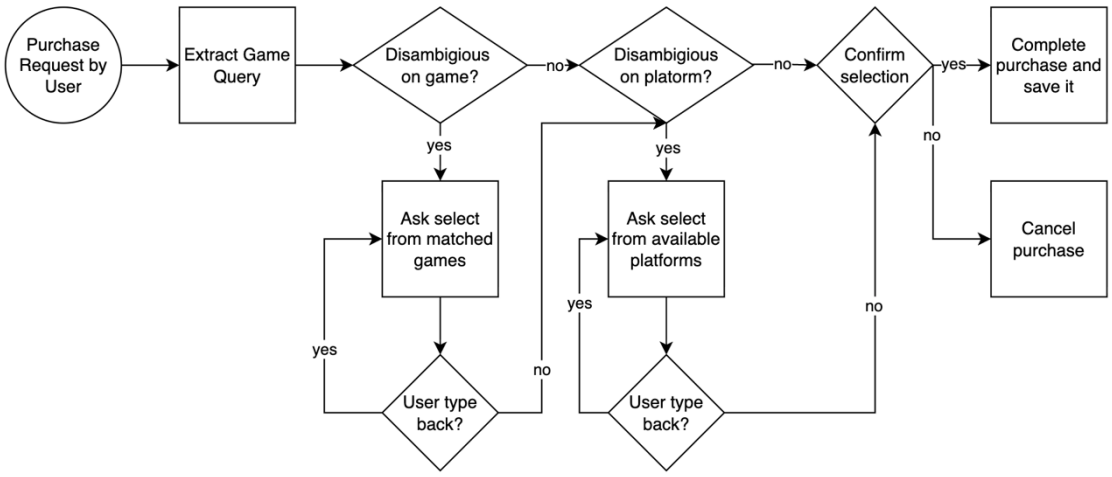


Figure 2: Game Purchase Workflow. The diagram illustrates the multi-step transaction process including disambiguation, platform selection, confirmation, and back-navigation logic.

2.2 Implementation

2.2.1 Intent Engine

The intent engine (`intent_engine.py`) provides intent classification using TF-IDF similarity. Intent examples are loaded from `intents.json`. The `match()` function vectorises both user input and all examples using `TfidfVectorizer(ngram_range=(1,2))` and computes cosine similarity via `cosine_similarity()` from scikit-learn. The highest-scoring intent label is returned along with its similarity score and matched example. A base threshold (0.50) ensures low-confidence inputs return *None* as the intent label.

2.2.2 Name engine

The name engine (`name_engine.py`) uses the `NameRecogniser` class to extract user names. The `normalise_intro()` function first fixes noisy or misspelled introduction phrases by tokenising the input and matching examples using `word_tokenize()` (NLTK) and reconstructing a correct template ("my naem is…" to "my name is…"). After normalisation, `extract_name()` applies a regex that matches 1—5 name tokens, including Unicode accents, hyphens, and apostrophes, following phrases such as "my name is". The extracted name is then capitalised using `w.capitalize()` to convert "john smith" to "John Smith".

2.2.3 QA Engine

The QA engine (qa_engine.py) retrieves answers from `qa_dataset.csv` via `load_qa_data()`. The `QaEngine` class builds a TF-IDF matrix by vectorising all stored questions using `TfidfVectorizer(ngram_range=(1,2), stop_words='english')` to filter common English stopwords. The `search()` method transforms user input via `vectorizer.transform()`, computes cosine similarity against the question matrix using `cosine_similarity()`, and returns the highest-scoring match via `np.argmax()`. The intent handler compares this QA similarity score against intent classification confidence to determine whether to return the QA answer or route to intent-based handling.

2.2.4 Games Engine and Detector

The games engine (`games_engine.py`) loads metadata from `games_data.csv` via `load_games_data()`, which includes title, platform, genre, publisher, developer, critic score, release date, and price. The `GamesEngine` class builds separate `TfidfVectorizer()` instances for titles, publishers, and developers, computing cosine similarity with a threshold of 0.40 for fuzzy matching (e.g., "FIFA" matching "FIFA 23"). Structured filtering uses comparison operators for platforms, genres, prices, and release years. The companion `games_detector.py` module extracts parameters using regex patterns and alias dictionaries from `config.py` (e.g., `PLATFORM_ALIASES = {"PS4" : ["playstation 4", "ps4"]}`), normalising queries like "playstation 4 games" to "PS4 games" before invoking filter methods such as `filter_by_platform()`.

2.2.5 NLG Component

Natural language generation is implemented in the `responses.py` module using f-string templates. Each intent or dialogue state maps to a dedicated function (e.g., `greeting()`, `confirm_order()`, `games_by_platform()`) that constructs a response by interpolating runtime variables into f-string literals. For example, `greeting(name)` contains `f"Hi {user_name.value}! Nice to meet you."`, producing `"Hi John! Nice to meet you."` when called with `name="John"`. Intent handlers pass context variables (user names, selected games, platforms, prices) as function arguments. The `UserNameProxy` class uses a `@property` decorator for dynamic name retrieval. When templates access `user_name.value`, they loads the current name from storage or defaults to `"user"`, enabling consistent personalisation without explicit parameter passing.

2.2.6 Storage

The storage module (`storage.py`) manages dataset loading and user data persistence. CSV datasets are loaded using `pandas.read_csv()` and converted to Python lists via `.values.tolist()` for use by the QA and games engines. For user-specific information, such as the name, name preference, purchase and search history, the module uses `joblib.dump()` to save and `joblib.load()` to load small Python objects.

2.2.7 Intent Handlers

The main intent handler operates as the chatbot's top-level router and dialogue state controller. In `handle_intent()`, it normalises user input and invokes `intent_engine.match()` and `qa_engine.search()`. If the QA similarity os higher, it yields a QA response; otherwise, game-related intents are forwarded to the `GamesHandler`, and all remaining intents are handled by `_handle_non_game_mode()`. Two confidence thresholds (0.40 and 0.70) are applied: very low scores trigger a generic fallback response, while intermediate scores produce a clarification prompt that paraphrases the closest example.

`_handle_non_game_mode()` processes greetings, help, small talk and emotional follow-ups, identity management, and the goodbye flow. It uses state flags to track name availability, explicit and implicit name-change operations, the goodbye flow, and pending emotional responses. These flags support correct handling of "yes/no/back" inputs, confirmation of name changes or removals, and decision on whether the name should be stored across sessions. The module interacts with the storage layer to persist or clear the user's name, name-saving preference, purchase history, and search history.

The `GamesHandler` manages all game-related intents via `handle_game_intent()`. It uses the games engine and detector utilities to convert free-text utterances into structured queries for search and filtering, and maintains multi-step interaction states, including game and platform selection, purchase confirmation, and back-navigation logic. It also records completed purchases in the storage module, updating the user's purchase history accordingly.

2.3 Justification

2.3.1 Modular Architecture

A modular architecture was adopted to ensure that intent classification, QA retrieval, name handling, and game search function independently. Each component can be developed, tested, and modified in isolation, while the intent handler coordinates them at runtime. This separation of concerns makes the system easier to extend without breaking existing functionality.

2.3.2 Intent, QA and Game Modelling

TF-IDF with cosine similarity was chosen for intent detection, QA and game search because it performs well with small hand-written examples sets. Representing utterances, questions and game titles in vector space allows the system to handle paraphrased inputs without exact keyword matching, while keeping the models simple and deterministic. For intent and QA, unigrams and bigrams are used so that short multi-word patterns such as "how much" or "how are you" influence the match. In the QA engine, English stop words are removed to focus the similarity on content words rather than generic question phrases. In contrast, the games engine uses unigram vectors without stop-word removal so that all tokens in short game titles, publisher and developer names are preserved. Similarity thresholds were tuned empirically to balance rejecting unrelated inputs with correctly recognising paraphrased queries.

2.3.3 Input Understanding Components

Regex-based extraction for names, platforms, genres, and numeric values was chosen because these domains have constrained vocabularies with predictable patterns. Regular expressions provide exact control over matching behaviour and fail predictably, unlike learned models, which may generalise incorrectly to out-of-domain inputs.

2.3.4 Response Generation and Persistence

Template-based NLG ensures grammatically correct, predictable responses, which is critical for purchase confirmations and error messages, and avoids generative-model hallucinations. Joblib provides sufficient persistence for user state (name, purchase history, search history) without database overhead for a single-user prototype.

3 Conversational Design

3.1 Prompt Design

The system uses clear conversational markers and key phrases such as “Hi”, “Hello”, and “Goodbye” to signal turn changes and keep the dialogue natural. Prompts are deliberately short and precise, ensuring users always know what type of input is expected. For example, when showing a numbered list, the chatbot asks the user to “choose a number from the list above”, and confirmation steps use direct yes/no questions. Prompts are structured to support each task: browsing uses selection prompts, purchasing uses step-by-step questions, and clarification prompts restate the system’s best guess when intent confidence is low. This consistent, concise prompting helps guide the user smoothly through all interactions.

3.2 Discoverability

The chatbot provides contextual help throughout the interaction. When users request help, the system presents a structured list of capabilities with example queries, enabling users to quickly discover available features and how to phrase requests.

Discoverability is further supported through context-aware guidance. After an invalid input, the chatbot explains what actions are available next (e.g., “please answer ‘yes’ or ‘no’, or type ‘back’”). During browsing and purchasing, prompts are tailored to the current stage of the task, such as asking the user to select a game or platform by index, so that the next valid actions are clear.

The system also assists users by offering example-driven suggestions. For broad or ambiguous requests, such as “let me buy some games”, the chatbot highlights common search options (e.g., by genre, platform, or price), helping users to discover functionality naturally without needing to rely solely on the help command.

To maintain readability during exploration, the system applies progressive disclosure. When a filter returns a very large set of games, only up to 20 randomly selected results are shown. This approach keeps responses manageable while still providing a representative sample of available titles and prevents overwhelming the user.

3.3 Error Handling

Error handling in the chatbot relies on both the confidence scores produced by the intent engine and explicit validation of user input during multi-step interactions. When intent confidence falls below the low threshold, the chatbot treats the utterance as not understood, and returns a neutral clarification request, prompting the user to rephrase rather than making an unreliable guess. For intermediate confidence scores, the system proposes its best interpretation (e.g., “Are you trying to say, ‘let me buy some games’?”) and asks the user to confirm, providing lightweight disambiguation while maintaining safe conversational recovery.

Within the multi-step purchasing flow, the system validates inputs before advancing, for example, checking numbered selections and navigation commands. Out-of-range indices or unsupported inputs trigger targeted feedback that explains the issue and restates valid options, such as selecting a number from the list or typing “back” to return to the previous step.

The chatbot also manages unrelated inputs during multi-turn interactions, such as name removal, name changes, or the goodbye process. If the user provides an unrelated intent once, the system issues a reminder of the current task. If the user does so a second time, the chatbot automatically exits the ongoing multi-step interaction to prevent the dialogue from becoming stuck or confusing.

By combining confidence-based intent handling, step-specific validation, and rules for managing irrelevant inputs, the system offers clear recovery paths from misunderstandings and maintains a coherent conversational structure without interrupting the user abruptly.

3.4 Personalisation

Personalisation in the chatbot focuses on adapting its behaviour to the user. Once the user provides their name, for example, by saying "my name is John", the system updates all subsequent turns to use that name directly. In dialogue displays, the user label changes from the generic "You:" to the personalised form "John:", and the chatbot's responses also incorporate the name in greetings and acknowledgements (e.g., "Hi John! How can I help you today?"). This creates a more natural and human-like conversational style.

If the user chooses to save their name at the end of a session, the chatbot reuses it in future sessions and greets returning users with personalised messages such as "Welcome back to the GAMES SHOP, John!". Users maintain full control over this behaviour: they can change their name at any time, either implicitly by introducing a new one or explicitly via a dedicated intent, and they may also request the chatbot to delete their saved name, after which the chatbot returns to non-personalised greetings.

Personalisation also supports recommendations. When the user has recently purchased a game, the chatbot uses that purchase context, specifically the game's platform and genre, to suggest additional titles that align with the user's demonstrated preferences. This ensures that personalisation is not limited to name usage but also reflects the user's interests and past behaviour.

3.5 Confirmation

Confirmation strategies in the chatbot match the potential impact of errors. In the purchase flow, once the user selects a game and platform, the system presents a summary and asks for explicit yes/no confirmation before completing the order. The same approach is used for actions that modify or delete personal data, such as removing or changing the stored name, and the goodbye flow asks whether the name should be remembered for future sessions. The chatbot also applies confirmation during interpretation. When the intent classifier has medium confidence, it restates its best guess and asks the user to confirm, following the three-tier confirmation model.

3.6 Context Tracking

The chatbot remembers important details during the current conversation so multi-step tasks run smoothly. For instance, it keeps track of whether the user is in a purchase flow, which stage they are at, and which game or platform they have already chosen. This allows the system to continue the interaction naturally and generate summaries without asking the user to repeat information. The chatbot also supports a "back" command by returning to the previous step. If the user brings up something unrelated in the middle of a task, the chatbot reminds them of what they were doing, and repeated interruptions cause it to exit the flow to avoid confusion.

4 Evaluation

The system was evaluated using intent classification performance metrics and the Chatbot Usability Questionnaire (CUQ). Intent classification was assessed on a test set of 713 utterances covering all 49 intent categories across greetings, name handling, game exploration, filtering, purchasing, and conversational features. The classifier achieved 97.5% overall accuracy, with a weighted average F1-score of 0.975, precision of 0.980, and recall of 0.975.

Performance analysis revealed that 41 out of 49 intents achieved perfect or near-perfect F1-scores (≥0.95). Notable exceptions included `games_search_by_genre` (F1=0.667, recall=0.500) and `games_search_by_platform` (F1=0.741, precision=0.667), where ambiguous phrasing overlapped with similar intents. The weighted average outperformed the macro average (0.975 vs 0.956), indicating slightly stronger performance on high-frequency intents, which aligns with expected behaviour in imbalanced datasets.

System reliability was validated through structured testing of multi-turn dialogues. The game purchase workflow, including title disambiguation, platform selection, and confirmation stages, executed consistently without state management failures across 20 test scenarios. Edge cases such as "back" navigation, explicit name change flows, and history retrieval were tested to ensure robust error handling.

Usability was measured using the standardised 16-item CUQ [4], where scores are calculated by summing responses (1-5 scale) for odd-numbered positive and even-numbered negative questions separately, then applying:

$$CUQ = \frac{(\Sigma Odd - 8) + (40 - \Sigma Even)}{64} \times 100$$

The chatbot achieved a CUQ score of 92%, indicating strong usability for a rule-driven architecture.

5 Discussion

Evaluating this system through a Responsible Research and Innovation (RRI) framework reveals both notable strengths and important ethical considerations. The CUQ evaluation score demonstrates that the chatbot achieves good accessibility and usability, which supports RRI principles around inclusivity. However, the similarity-based intent classifier presents a limitation in handling ambiguous user inputs due to its probabilistic nature. While such errors are unlikely to cause harm within a game shop context, they underscore the value of transparency in system design.

Regarding data responsibility, the system implements a privacy-conscious approach by storing only optional user data (username, purchase and search history) and requiring explicit consent before saving information. Users can delete their data at any time, maintaining full control over what is retained. This data minimalism strategy effectively mitigates privacy risks.

The application domain itself presents low ethical risk, though care must be taken to avoid manipulative selling practices. The system's reliance on rule-based logic rather than ranking or recommendation algorithms ensures it remains neutral and non-exploitative. Overall, this analysis demonstrates that implementing RRI principles, particularly transparency, user autonomy, and data minimalism, provides a solid foundation for responsible development. Future iterations should prioritise strengthening system robustness to further enhance user confidence.

REFERENCES

[1] Keshav Agrawal. (2025). The Future of AI in Digital Search: Towards a Fully Conversational Experience. *Journal of Computer Science and Technology Studies*, *7*(4), 298-306. https://doi.org/10.32996/jcsts.2025.7.4.34

[2] Adam, M., Wessel, M. & Benlian, A. AI-based chatbots in customer service and their effects on user compliance. *Electron Markets* 31, 427–445 (2021). https://doi.org/10.1007/s12525-020-00414-7

[3] Zumstein, Darius & Hundertmark, Sophie. (2017). Chatbots – An Interactive Technology for Personalized Communication, Transactions and Services. IADIS International Journal on WWW/Internet. 15. 96-109.

[4] Samuel Holmes, Anne Moorhead, Raymond Bond, Huiru Zheng, Vivien Coates, and Michael Mctear. 2019. Usability testing of a healthcare chatbot: Can we use conventional methods to assess conversational user interfaces? In Proceedings of the 31st European Conference on Cognitive Ergonomics (ECCE '19). Association for Computing Machinery, New York, NY, USA, 207–214. https://doi.org/10.1145/3335082.3335094